

Planning an Oracle Implementation: EBS and Fusion, Side by Side

The choice of platform is already made. What matters now is getting the implementation strategy right — and that strategy shares a common backbone whether you're on E-Business Suite or Fusion, with the execution diverging in specific, costly-to-miss ways.

9 MIN READ · SUPPLY CHAIN CAPABILITIES

IN THIS DOCUMENT

1. The common backbone
2. If you're implementing EBS
3. If you're implementing Fusion
4. Similarities at a glance
5. Where they genuinely differ
6. Lessons from the field
7. How Insu Dynamics takes care of this, point to point

The common backbone

Strip away the platform, and every well-run Oracle implementation follows the same five stages. This is the part of the strategy that doesn't change based on which product you picked.

- 1 Discovery & fit-gap**
Map current-state processes against the target system's standard capability and log every gap — this document drives every decision after it, on either platform.
- 2 Solution design**
Translate approved gaps into a build plan: configuration, extension, or integration, with an explicit owner and effort estimate for each.
- 3 Configuration & build**
Build in iterations tied to business processes (order-to-cash, plan-to-produce), not technical modules, so testing can validate a real process end to end.
- 4 Testing**
Unit, system integration, and user acceptance testing, in that order, with defects triaged against the original fit-gap log so nothing silently drops.
- 5 Deployment & stabilization**
Go-live is the midpoint of the project, not the end. Budget a defined hypercare period with the implementation team still engaged.

If you're implementing EBS

EBS implementations are shaped by the fact that you own the full stack — code, infrastructure, and upgrade timing.

- **Customization is a first-class tool, not a last resort.** Forms personalization, PL/SQL extensions, and custom reports (Reports 6i/BI Publisher) are normal ways to close gaps, so the design phase must budget real development time, not just configuration time.
- **Infrastructure is part of the project plan.** Environment provisioning, sizing, and patching schedules sit inside your critical path, not a vendor's.
- **Regression testing runs long.** Every customization is a thing that can break on the next patch, so the testing cycle is proportional to how much you've built, not just what you've configured.
- **Integrations tend to be point-to-point.** SQL*Loader, custom PL/SQL APIs, and file-based interfaces are common — durable, but each one is a maintenance line item later.

If you're implementing Fusion

Fusion implementations are shaped by the opposite constraint: you don't own the stack, and the platform changes under you on a schedule.

- **Configuration comes before extension, and extension before customization.** The design phase should actively resist the instinct to rebuild an EBS customization one-for-one in Fusion — most gaps close with setup, not code.
- **Reporting moves to OTBI and BI Publisher.** Legacy custom reports don't carry over; rebuilding the reporting layer is real scope, not an afterthought.
- **Quarterly releases are a standing work stream.** Testing isn't a one-time project phase — it becomes a recurring calendar item for as long as you run the system.
- **Integration shifts to OIC and REST/SOAP APIs.** This is generally faster to build than EBS-style interfaces, but it requires a different skill set on the team.

CAVEAT

Staffing an EBS-experienced team on a Fusion project without retraining them is one of the most common causes of scope creep — they will reach for customization by habit, on a platform built to discourage it.

Similarities at a glance

WORK STREAM	EBS	FUSION
Fit-gap discipline	Required	Required
Data migration strategy	Required	Required
Change management & training	Required	Required
Governance / steering committee	Required	Required
Defined hypercare period	Required	Required

Where they genuinely differ

DIMENSION	EBS APPROACH	FUSION APPROACH
Closing gaps	Custom code (Forms, PL/SQL)	Configuration, then low-code extension
Release cadence	Customer-controlled patching	Oracle-driven quarterly updates
Reporting	Custom reports / Discoverer	OTBI / BI Publisher

Infrastructure ownership	Customer-managed	Oracle-managed
Testing cycle	Long, customization-driven	Shorter, but recurring every quarter

Lessons from the field

NUGGET

The fit-gap workshop format barely changes between an EBS and a Fusion project — the questions you ask the business are the same. It's what you do with the answers that diverges.

TIP

Start the reporting conversation in week one on any Fusion project, regardless of platform experience. The decision between OTBI and custom BI Publisher reports quietly shapes a lot of downstream design.

The implementations that go smoothly, on either platform, are the ones where the team plans for the platform's actual operating model instead of the one they're used to. On EBS, that means budgeting for the code you own. On Fusion, it means budgeting for the releases you don't control. Treating either platform like the other is where timelines and budgets quietly slip.

INSU DYNAMICS

How we take care of this, point to point

Every risk raised in this document maps to something we actively manage on our engagements — not as a general promise, but line by line against what actually derails implementations.

Fit-gap workshops that don't translate into a real build plan

We run fit-gap as a structured, platform-aware exercise from day one — the output is a scoped, owned, estimated backlog, not a workshop deck.

EBS customization scope quietly growing during build

We size customization effort up front and track it against the fit-gap log throughout build, so it's a managed line item, not a surprise.

Fusion teams defaulting to EBS-style customization habits

We enforce a configuration-first design review at every gap decision, so Fusion projects stay inside the operating model that keeps them cost-efficient.

Reporting treated as an afterthought

We scope the OTBI / BI Publisher reporting layer in week one, alongside the core fit-gap, not after go-live pressure forces it.

Quarterly Fusion releases catching teams off guard

We set up a standing quarterly regression cadence as part of ongoing support, so new releases are validated, not discovered.

Go-live treated as the finish line

We stay engaged through a defined hypercare window, with the implementation team still on point until the system is genuinely stable.

This is what **innovation drives sustainability** means in practice for us — not a slogan, but a commitment to closing the exact gaps that cause Oracle implementations to slip, one point at a time.